

MODULE SÉCURITÉ DES DONNÉES

RAPPORT D'AUTOMATISATION CI/CD

Intégration continue avec Jenkins et GitHub: Build, SCM, Webhook et notifications par e-mail

Tuteur	Aicha WAR
Étudiant	Papa Ndiaga Diouf
Étudiant	Serigne Saliou Mbacké Seck
Étudiant	Therno Hamadou Tidian Tioubou
Date	30 juillet 2026
Environnement	Jenkins (Docker) GitHub Ngrok

Table des matières

1. Introduction	3
1.1 Objectifs pédagogiques	3
2. Méthodologie et environnement	3
3. Étape 1 : Build Jenkins avec un script statique	4
3.1 Contexte	4
3.2 Démarche	4
3.3 Résultat	5
4. Étape 2 : Intégration du code source depuis GitHub	7
4.1 Contexte	7
4.2 Authentification Jenkins ↔ GitHub	7
4.3 Configuration du pipeline en mode SCM	8
4.4 Résultat	10
5. Étape 3 : Déclenchement automatique par Webhook GitHub	12
5.1 Activation du déclencheur côté Jenkins	12
5.2 Exposition de Jenkins via ngrok	12
5.3 Configuration du Webhook côté GitHub	13
5.4 Test du déclenchement automatique	13
5.5 Comparatif Webhook GitHub vs SCM Polling	15
6. Étape 4 : Notification par e-mail	16
6.1 Contexte	16
6.2 Configuration des identifiants et de l'hôte Jenkins	16
6.3 Configuration du serveur SMTP	17
6.4 Test de la configuration	20
6.5 Intégration dans le pipeline et réception du rapport de build	20
6.6 Résultat	23
7. Conclusion	24

1. Introduction

L'intégration continue (CI) consiste à automatiser la vérification du code à chaque modification apportée par les développeurs, afin de détecter au plus tôt les erreurs, les régressions ou les vulnérabilités. Jenkins est l'un des outils d'automatisation les plus utilisés pour mettre en place ce type de pipeline : il permet de définir, sous forme de script, une suite d'étapes (récupération du code, analyse, tests, notification) déclenchées automatiquement.

Ce rapport documente la mise en place progressive d'une chaîne d'intégration continue autour de l'application vulnérable OWASP Juice Shop, utilisée ici comme projet support pour exécuter une analyse statique de sécurité du code (SAST) avec l'outil Semgrep. La démarche a été construite par étapes successives, chacune ajoutant un niveau d'automatisation supplémentaire :

- **Étape 1** : Réalisation d'un premier build Jenkins à partir d'un script statique, sans lien avec un dépôt de code source.
- **Étape 2** : Intégration du code source depuis un dépôt GitHub, pour que Jenkins récupère automatiquement la version à jour du projet.
- **Étape 3** : Mise en place d'un Webhook GitHub, afin que chaque modification poussée sur le dépôt déclenche automatiquement un nouveau build, sans intervention manuelle.
- **Étape 4** : Mise en place d'un système de notification par e-mail, informant automatiquement de l'issue de chaque build.

1.1 Objectifs pédagogiques

- Comprendre le fonctionnement d'un pipeline Jenkins et la différence entre un script statique et un pipeline piloté par SCM (Source Control Management).
- Configurer une intégration sécurisée entre Jenkins et un dépôt GitHub privé (jetons d'accès, credentials).
- Mettre en place un déclenchement automatique de build par Webhook. Bonus, comprendre les contraintes réseau associées (exposition d'un service local via un tunnel).
- Configurer un système de notification automatique par e-mail à l'issue d'un build.

2. Méthodologie et environnement

Jenkins a été déployé localement dans un conteneur Docker (image devops-infra-jenkins-jenkins), exposé sur le port 8080 pour l'interface web et le port 50000 pour la communication avec les agents. Le projet utilisé comme cible d'analyse est le dépôt de code source de l'application OWASP Juice Shop.

L'outil d'analyse statique de sécurité **Semgrep** a été utilisé au sein de chaque pipeline, exécuté lui-même dans un conteneur Docker, afin de scanner le code source à la recherche de vulnérabilités connues. Le dépôt GitHub utilisé pour héberger le code et le pipeline (Jenkinsfile) est un dépôt privé, ce qui a nécessité la mise en place d'une authentification par jeton d'accès personnel.

Pour la partie automatisation par **Webhook**, Jenkins étant hébergé en local (non accessible depuis Internet), l'outil **ngrok** a été utilisé pour créer un tunnel HTTPS temporaire exposant publiquement l'interface Jenkins, condition nécessaire pour que GitHub puisse notifier Jenkins lors d'un push.

3. Étape 1 : Build Jenkins avec un script statique

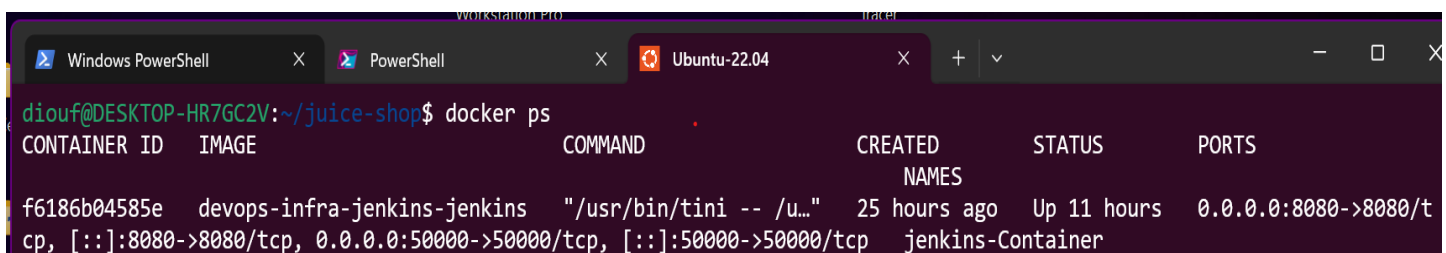
Premier build Jenkins

Outils : Jenkins (Pipeline script), Docker, Semgrep

Objectif : Réaliser un premier build fonctionnel à partir d'un script de pipeline saisi directement dans Jenkins, sans lien avec un dépôt de code source distant.

3.1 Contexte

Avant d'automatiser la récupération du code depuis un dépôt distant, la première étape consiste à valider que Jenkins est correctement installé et opérationnel, et qu'il est capable d'exécuter une tâche d'analyse de sécurité de bout en bout. Le conteneur Jenkins a d'abord été vérifié comme étant actif.

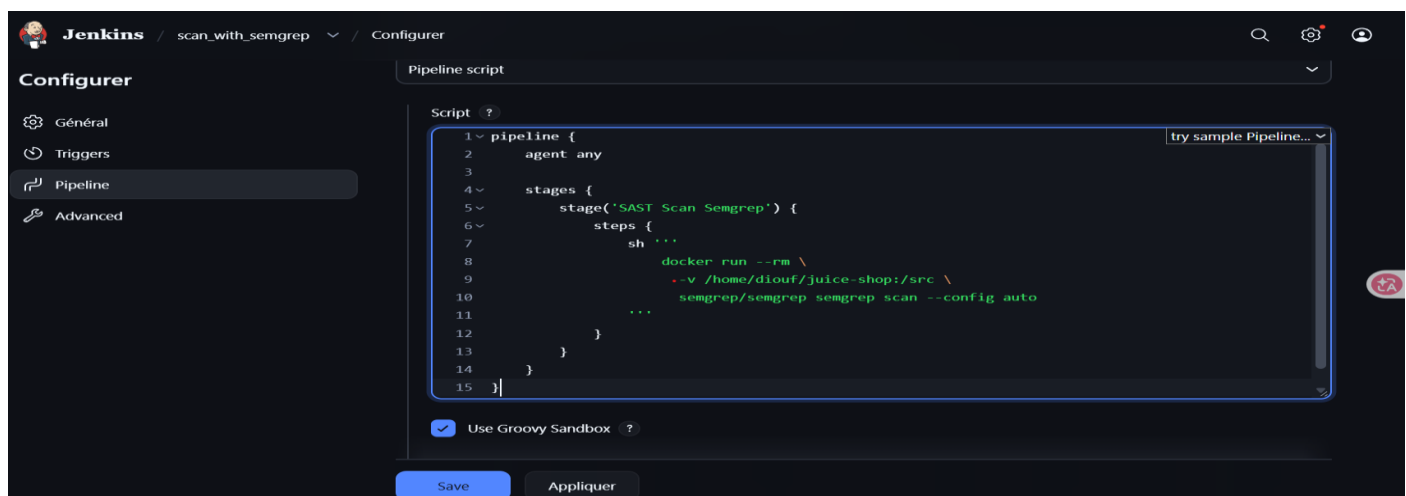


```
diouf@DESKTOP-HR7GC2V:~/juice-shop$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
f6186b04585e   devops-infra-jenkins-jenkins        "/usr/bin/tini -- /u..." 25 hours ago   Up 11 hours   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp
```

Capture 1 : Commande `docker ps` confirmant que le conteneur Jenkins (jenkins-Container) est actif, avec les ports 8080 et 50000 exposés.

3.2 Démarche

Un job de type Pipeline, nommé `scan_with_semgrep`, a été créé dans Jenkins. Contrairement à un pipeline piloté par SCM, le script de ce pipeline est saisi manuellement (« Pipeline script ») directement dans la configuration du job : il est donc qualifié de « statique », car il ne provient d'aucun dépôt versionné et doit être modifié à la main en cas de changement. Ce script définit une seule étape, qui exécute Semgrep dans un conteneur Docker, en montant un dossier local du poste de travail (`/home/diouf/juice-shop`) comme volume source à analyser.



Capture 2 : Configuration du pipeline `scan_with_semgrep` : script statique saisi manuellement, montant le dossier local `/home/diouf/juice-shop` et lançant un scan Semgrep avec la configuration automatique.

```

pipeline {
  agent any

  stages {
    stage('SAST Scan Semgrep') {
      steps {
        sh '''
          docker run --rm \
            -v /home/diouf/juice-shop:/src \
            semgrep/semgrep semgrep scan --config auto
        '''
      }
    }
  }
}

```

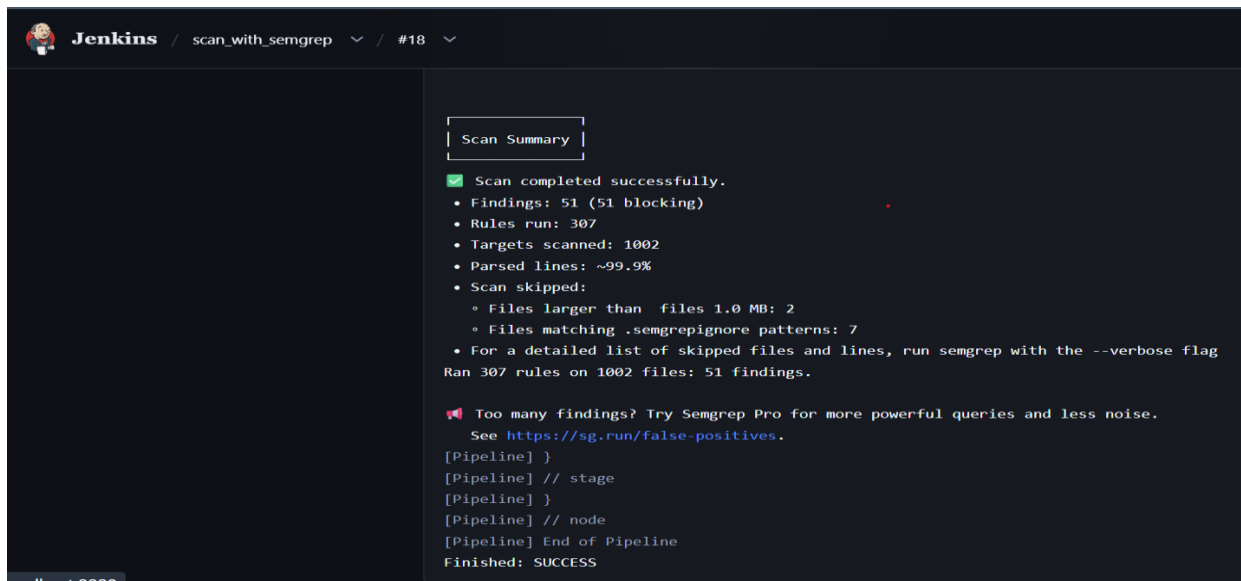
Le pipeline a ensuite été lancé manuellement depuis l'interface Jenkins.



Capture 3 : Historique des builds du job scan_with_semgrep : le build #18 s'est terminé avec succès.

3.3 Résultat

La console de build confirme la bonne exécution du scan de sécurité : Semgrep a analysé le code monté en volume et produit un résumé des vulnérabilités détectées, avec un statut final de succès.



```
Jenkins / scan_with_semgrep / #18  
  
[ Scan Summary ]  
  
✅ Scan completed successfully.  
• Findings: 51 (51 blocking)  
• Rules run: 307  
• Targets scanned: 1002  
• Parsed lines: ~99.9%  
• Scan skipped:  
  ◦ Files larger than files 1.0 MB: 2  
  ◦ Files matching .semgrepignore patterns: 7  
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag  
Ran 307 rules on 1002 files: 51 findings.  
  
🚩 Too many findings? Try Semgrep Pro for more powerful queries and less noise.  
See https://sg.run/false-positives.  
  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline  
Finished: SUCCESS
```

Capture 4 : Console du build #18 : résumé du scan (51 findings sur 1002 cibles analysées, 307 règles exécutées), build terminé avec le statut SUCCESS.

Cette première étape valide le bon fonctionnement de Jenkins et de l'intégration avec Semgrep, mais présente une limite importante : le code analysé est celui présent localement sur la machine hébergeant Jenkins, avec un chemin écrit en dur dans le script. Toute modification du code source nécessite une intervention manuelle, aussi bien pour la mise à jour du code que pour le déclenchement du build. L'étape suivante vise à lever cette limite en connectant Jenkins directement à un dépôt GitHub.

4. Étape 2 : Intégration du code source depuis GitHub

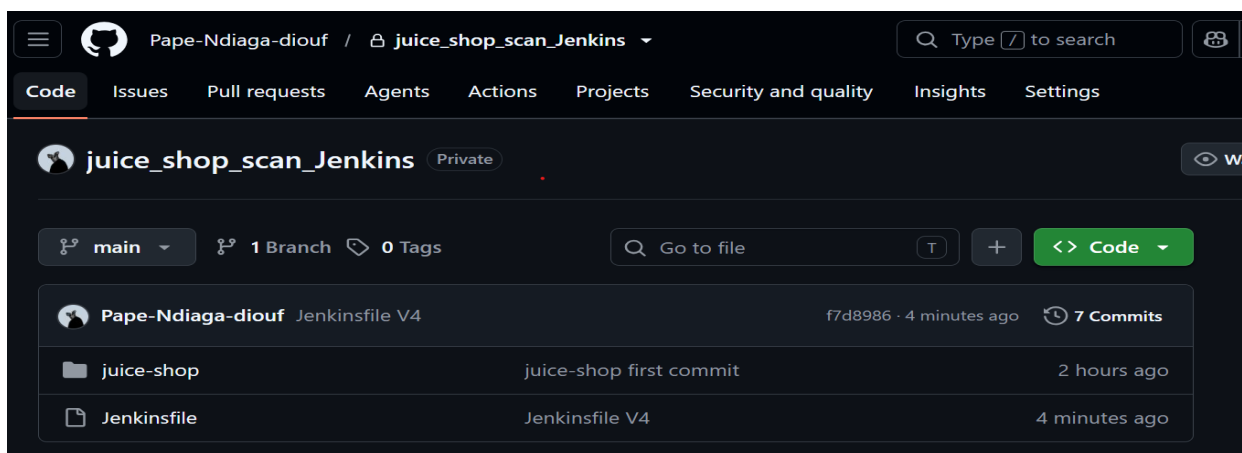
Pipeline piloté par SCM (GitHub)

Outils : Jenkins (Pipeline from SCM), GitHub, Git, Personal Access Token

Objectif : Faire récupérer automatiquement par Jenkins le code source et le script de pipeline depuis un dépôt GitHub, plutôt que de dépendre d'un script statique local.

4.1 Contexte

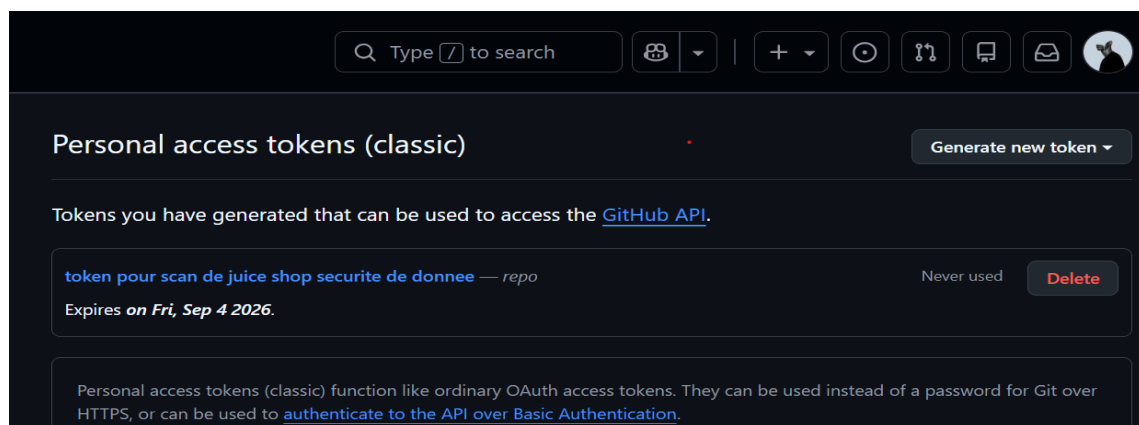
Pour que Jenkins puisse construire le projet à partir de sa version la plus à jour, le code source de Juice Shop ainsi que le script de pipeline (désormais formalisé dans un fichier Jenkinsfile versionné) ont été hébergés dans un dépôt GitHub privé.



Capture 5 : Dépôt GitHub privé juice_shop_scan_Jenkins, contenant le code source et le fichier Jenkinsfile versionné.

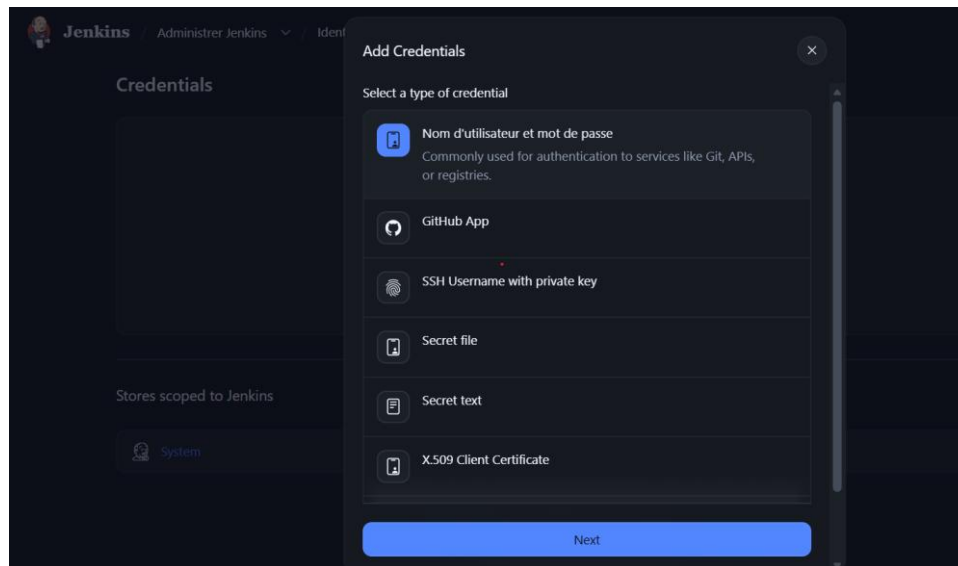
4.2 Authentification Jenkins ↔ GitHub

Le dépôt étant privé, Jenkins doit s'authentifier pour pouvoir le cloner. Un jeton d'accès personnel (Personal Access Token) a été généré depuis les paramètres du compte GitHub, avec les droits nécessaires sur le dépôt.

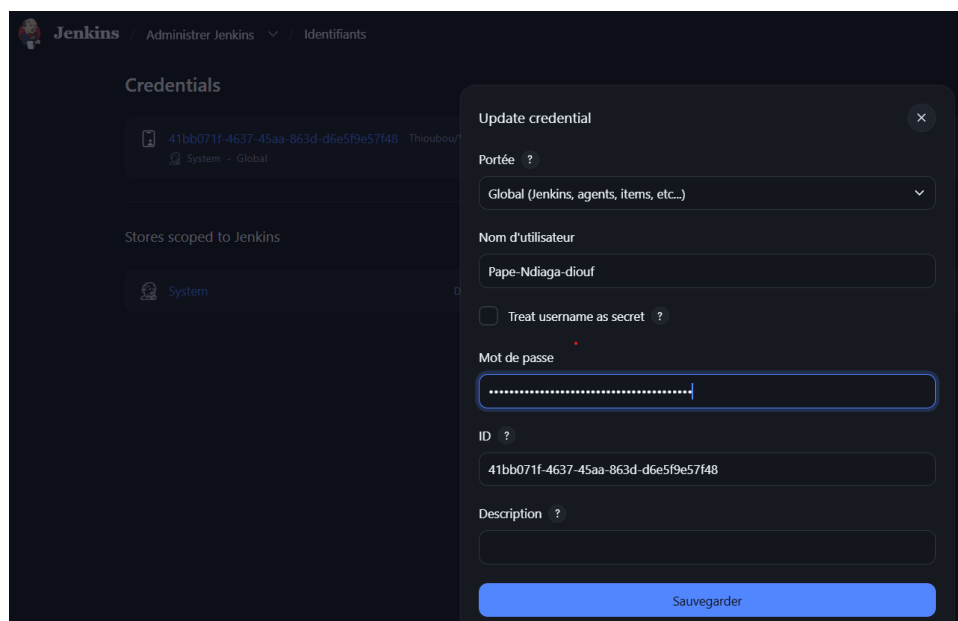


Capture 6 : Génération d'un jeton d'accès personnel (classic) sur GitHub, dédié à l'accès de Jenkins au dépôt.

Ce jeton a ensuite été enregistré dans le gestionnaire d'identifiants (Credentials) de Jenkins, sous la forme d'un couple nom d'utilisateur / mot de passe, le jeton étant utilisé à la place du mot de passe réel du compte GitHub.



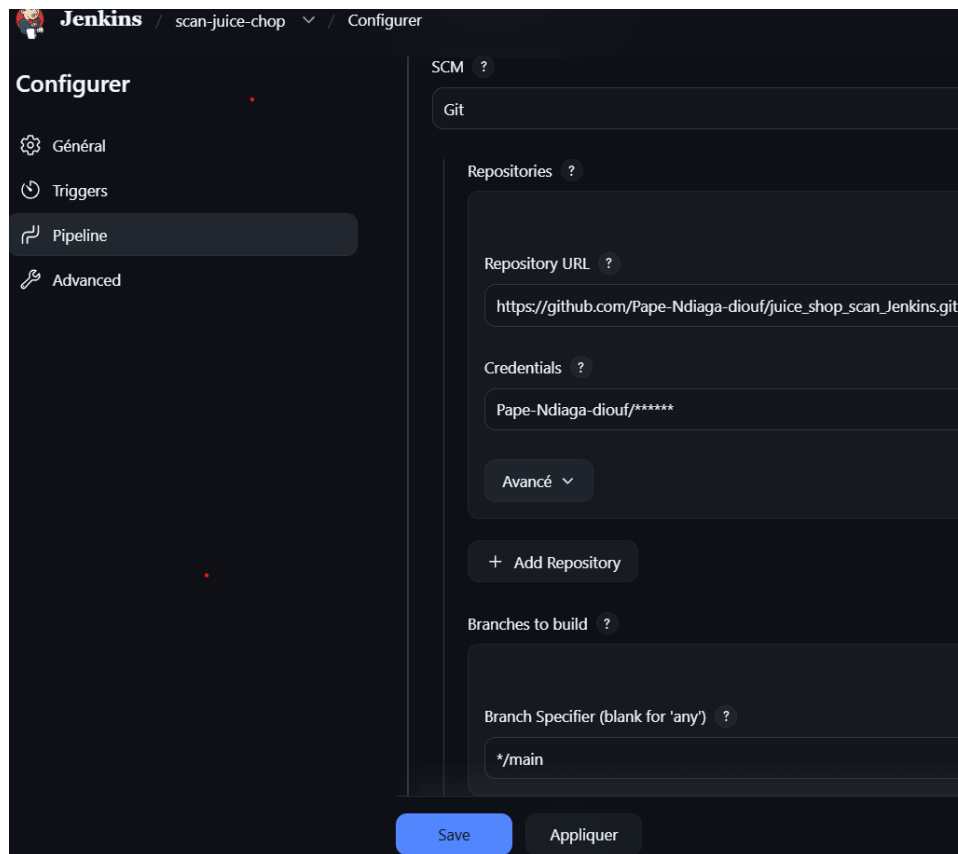
Capture 7 : Ajout d'un identifiant dans Jenkins



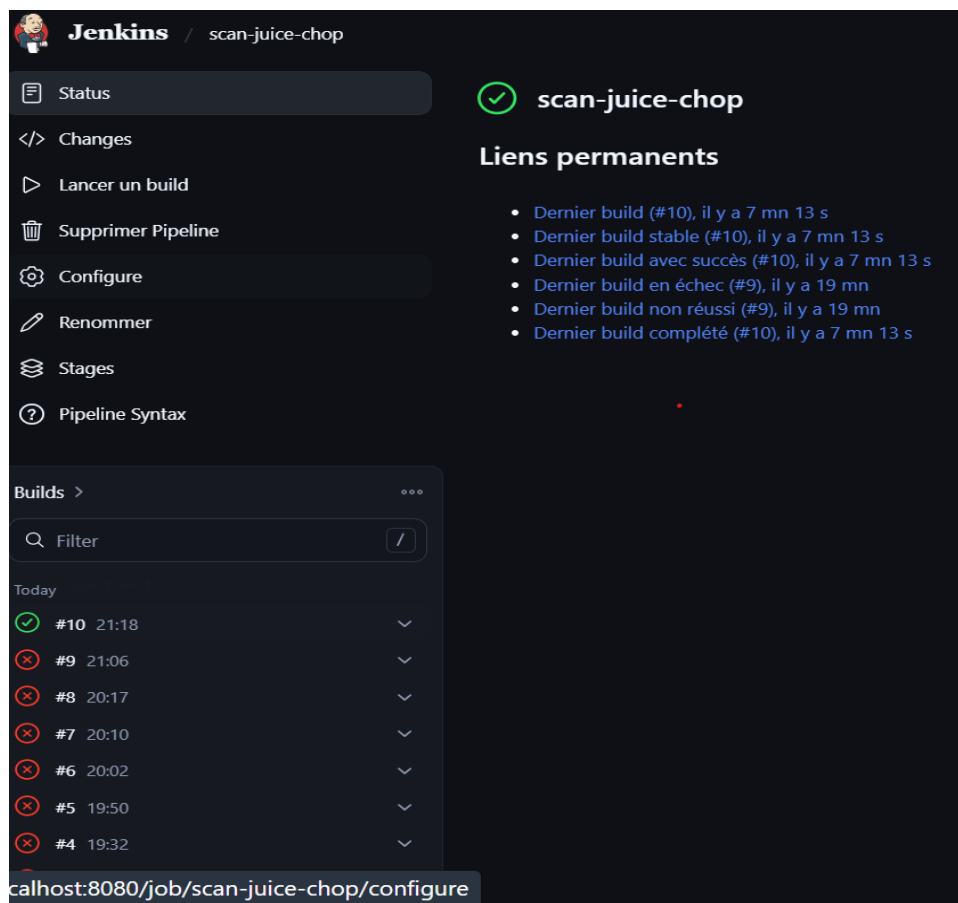
Capture 8 : Identifiant configuré avec le nom d'utilisateur GitHub et le jeton d'accès personnel comme mot de passe.

4.3 Configuration du pipeline en mode SCM

Un nouveau job, scan-juice-chop, a été configuré pour récupérer son code directement depuis le dépôt Git : URL du dépôt, identifiants créés précédemment, et branche à construire (* /main).



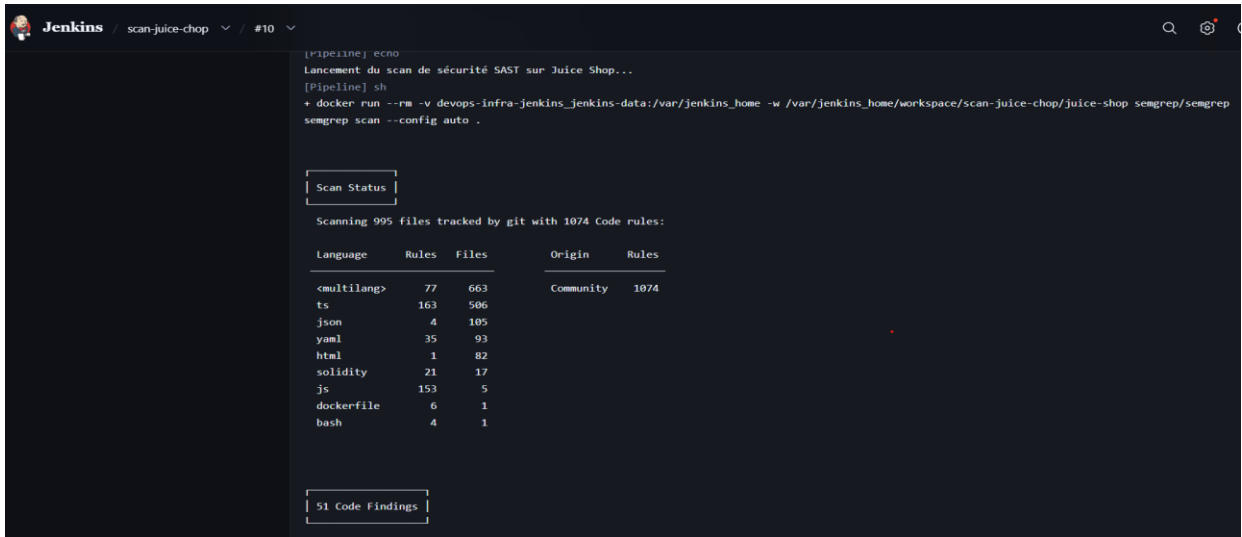
Capture 9 : Configuration SCM du job scan-juice-chop : dépôt Git, URL GitHub, identifiants, branche main.



Capture 10 : Historique des builds du job scan-juice-chop : le premier build réussi (#10).

4.4 Résultat

Le build réussi confirme que Jenkins clone désormais correctement le dépôt GitHub avant de lancer l'analyse Semgrep, cette fois sur l'ensemble des fichiers suivis par Git dans le dépôt (995 fichiers, 1074 règles au total, réparties par langage).



```
[Pipeline] echo
Lancement du scan de sécurité SAST sur Juice Shop...
[Pipeline] sh
+ docker run --rm -v devops-infra-jenkins-jenkins-data:/var/jenkins_home -w /var/jenkins_home/workspace/scan-juice-chop/juice-shop semgrep/semgrep
semgrep scan --config auto .
```

Scan Status

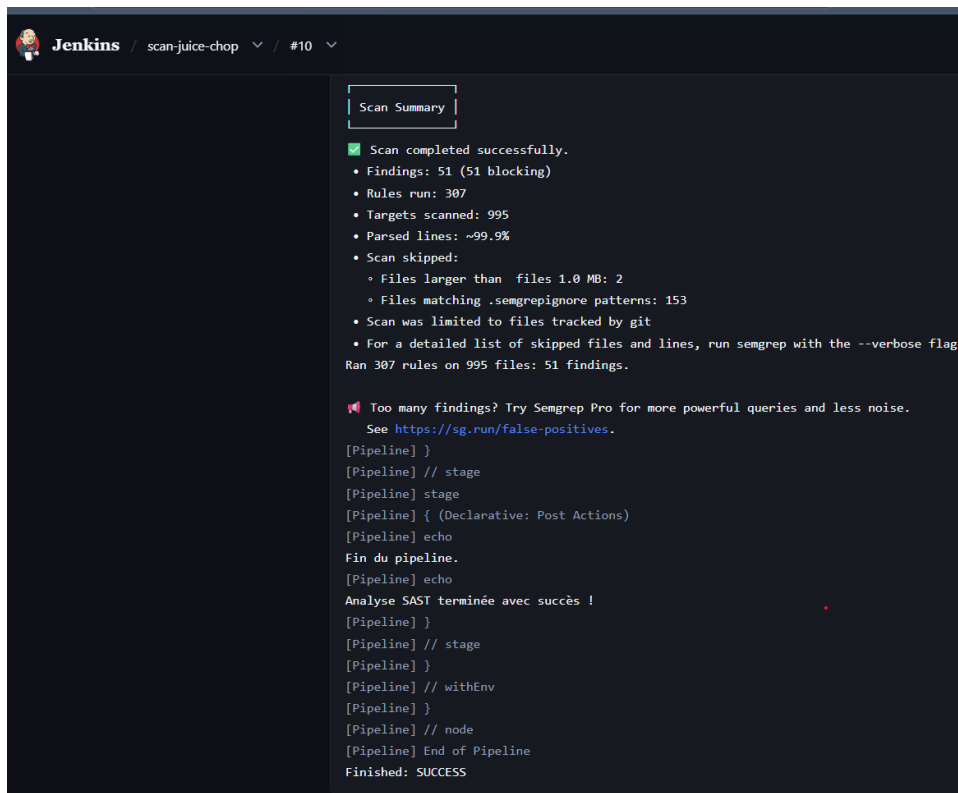
Scanning 995 files tracked by git with 1074 Code rules:

Language	Rules	Files	Origin	Rules
<multilang>	77	663	Community	1074
ts	163	506		
json	4	105		
yaml	35	93		
html	1	82		
solidity	21	17		
js	153	5		
dockerfile	6	1		
bash	4	1		

51 Code Findings

Capture 11 : Console du build #10

Le résumé final confirme la réussite du scan et du build, avec 51 anomalies détectées, cohérent avec le résultat obtenu lors du build statique de l'étape précédente, mais cette fois à partir d'un code source récupéré automatiquement depuis GitHub.



Scan Summary

- ✓ Scan completed successfully.
- Findings: 51 (51 blocking)
- Rules run: 307
- Targets scanned: 995
- Parsed lines: ~99.9%
- Scan skipped:
 - Files larger than files 1.0 MB: 2
 - Files matching .semgrepignore patterns: 153
- Scan was limited to files tracked by git
- For a detailed list of skipped files and lines, run semgrep with the --verbose flag

Ran 307 rules on 995 files: 51 findings.

🔊 Too many findings? Try Semgrep Pro for more powerful queries and less noise.
See <https://sg.run/false-positives>.

```
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Fin du pipeline.
[Pipeline] echo
Analyse SAST terminée avec succès !
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Capture 12 : Résumé final du build #10 : Analyse SAST terminée avec succès ! ».

Cette étape élimine la dépendance à un chemin de fichier local : Jenkins travaille désormais directement sur le code versionné dans GitHub. Il reste cependant nécessaire de déclencher chaque build manuellement depuis l'interface Jenkins, ce que l'étape suivante vise à automatiser.

5. Étape 3 : Déclenchement automatique par Webhook GitHub

Automatisation du déclenchement de build

Outils : Jenkins (GitHub hook trigger), GitHub Webhooks, ngrok

Objectif : Déclencher automatiquement un build Jenkins à chaque modification (push) détectée sur le dépôt GitHub, sans action manuelle.

5.1 Activation du déclencheur côté Jenkins

La première condition pour qu'un Webhook GitHub puisse déclencher un build est que le job Jenkins soit configuré pour accepter ce type de déclenchement. L'option « GitHub hook trigger for GITScm polling » a été activée dans la configuration du job.



Capture 13 : Activation du déclencheur « GitHub hook trigger for GITScm polling » dans la configuration du job scan-juice-chop.

5.2 Exposition de Jenkins via ngrok

Jenkins étant hébergé en local (accessible uniquement sur le réseau de la machine hôte), GitHub ne peut pas lui envoyer de notification directement : un serveur GitHub externe ne peut pas atteindre une adresse localhost. L'outil ngrok a été utilisé pour créer un tunnel sécurisé exposant temporairement le port local 8080 de Jenkins sur une URL publique en HTTPS.

```
diouf@DESKTOP-HR7GC2V:~$ ngrok --version
ngrok version 3.39.10
diouf@DESKTOP-HR7GC2V:~$
```

Capture 14 : Vérification de l'installation de ngrok (version 3.39.10).

Le tunnel a ensuite été lancé, générant une URL publique temporaire redirigeant vers le service Jenkins local.

```

ngrok

One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status      online
Account             Pape Ndiaga (Plan: Free)
Version             3.39.10
Region              Europe (eu)
Latency              93ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://cupcake-brewery-bobsled.ngrok-free.dev -> http://localhost:8080

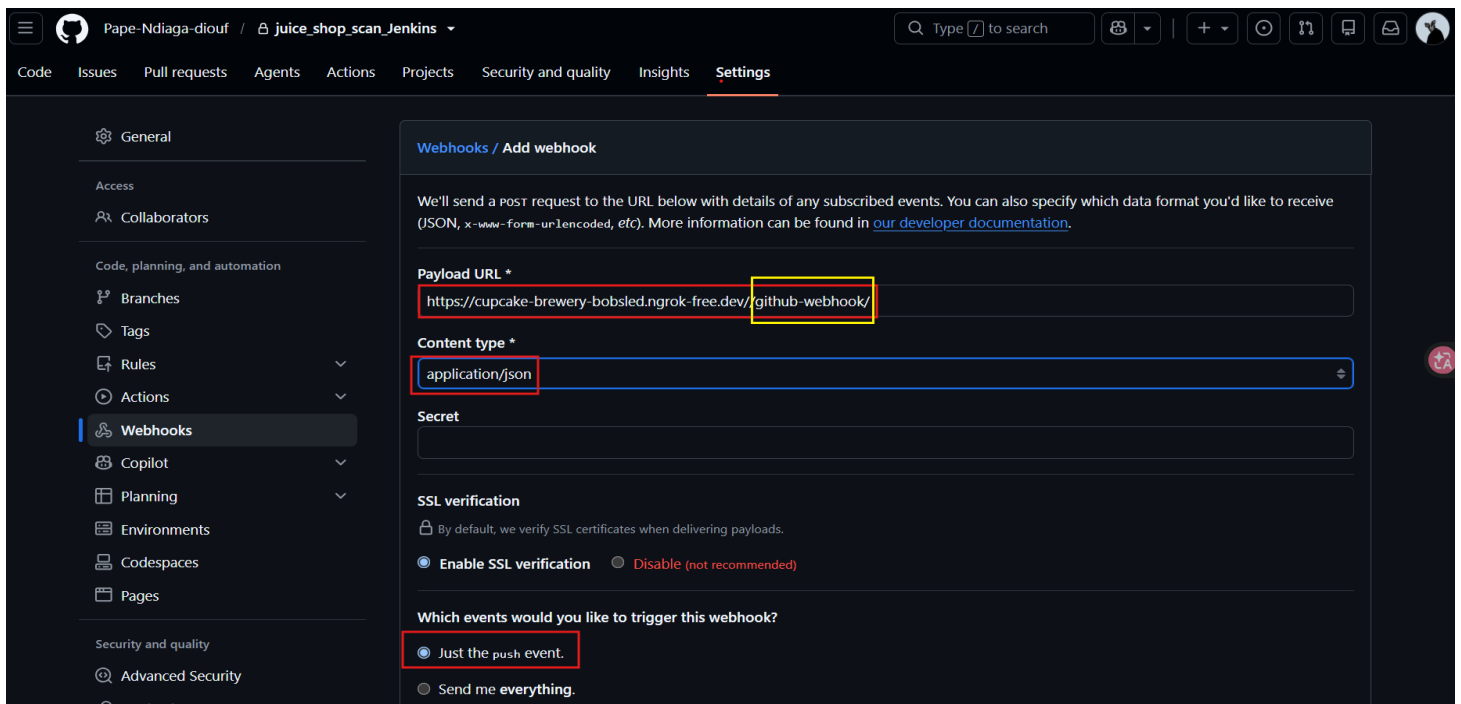
Connections
  ttl    opn    rt1    rt5    p50    p90
   0      0     0.00  0.00  0.00  0.00

```

Capture 15 : Tunnel ngrok actif : l'URL publique `https://cupcake-brewery-bobsled.ngrok-free.dev` redirige vers `http://localhost:8080` (Jenkins).

5.3 Configuration du Webhook côté GitHub

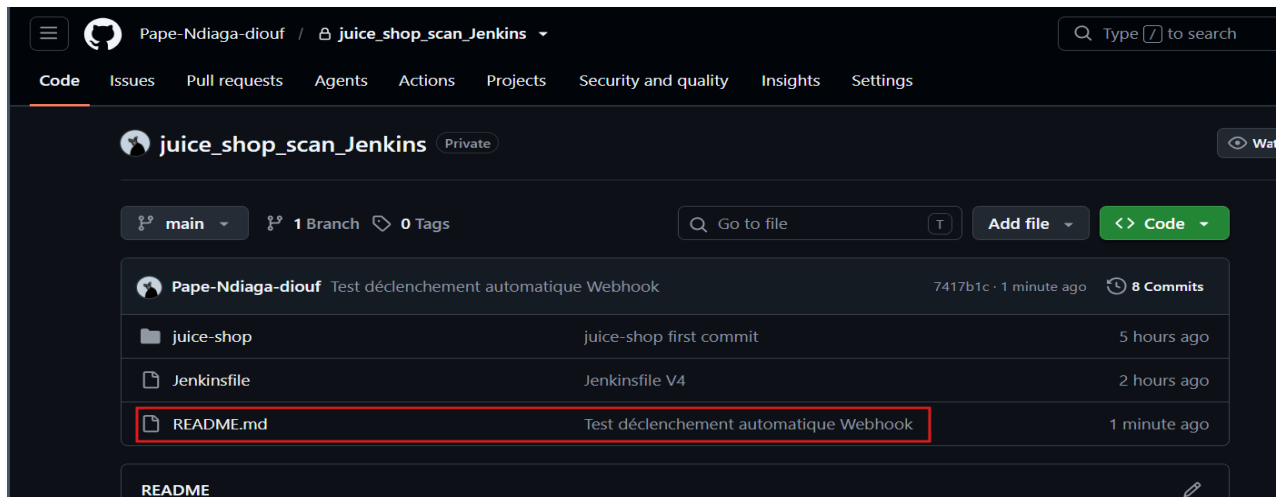
Dans les paramètres du dépôt GitHub, un Webhook a été ajouté, pointant vers l'URL publique fournie par ngrok, suivie du chemin standard attendu par le plugin GitHub de Jenkins (**/github-webhook/**). Le type de contenu a été fixé à **application/json**, et seul l'événement push a été sélectionné comme déclencheur, conformément au besoin (déclencher un build à chaque modification du code, sans notifier Jenkins pour d'autres types d'événements comme les issues ou les pull requests).



Capture 16 : Configuration du Webhook GitHub : URL de charge utile pointant vers le tunnel ngrok + `/github-webhook/`

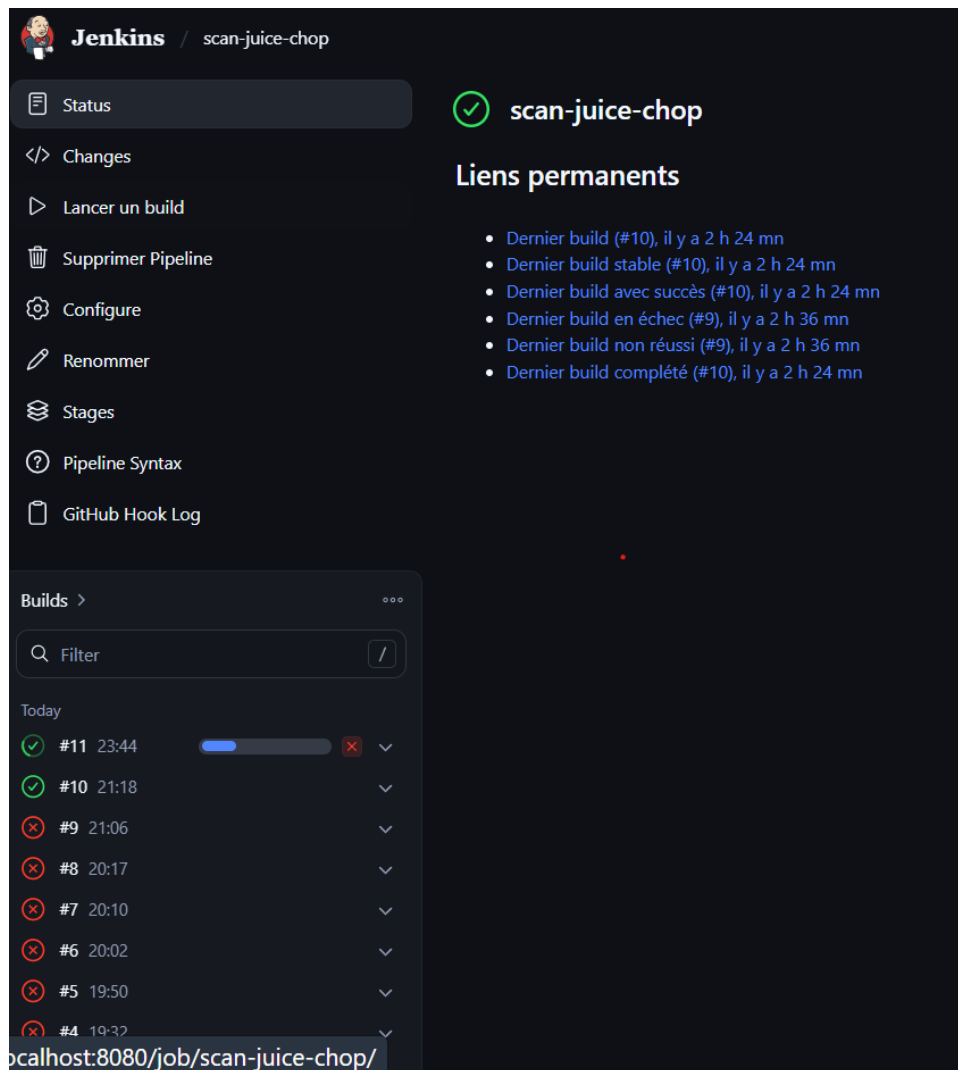
5.4 Test du déclenchement automatique

Pour valider la chaîne complète, une modification a été poussée sur le dépôt (ajout d'un fichier **README.md**), simulant une contribution réelle d'un développeur.



Capture 17 : Commit de test

Immédiatement après ce push, un nouveau build a démarré automatiquement dans Jenkins, sans aucune action manuelle. L'apparition d'une nouvelle entrée « **GitHub Hook Log** » dans le menu du job confirme que la notification du Webhook a bien été reçue et traitée par Jenkins.



Capture 18 : Le build #11 démarre automatiquement suite au push ; l'entrée « **GitHub Hook Log** » apparaît désormais dans le menu du job, confirmant la réception du Webhook.

Le build déclenché automatiquement s'est déroulé normalement jusqu'à son terme, avec un scan de sécurité complet et un statut final de succès, exactement comme lors d'un déclenchement manuel.

```

Jenkins / scan-juice-chop / #11
[Pipeline] Scan Summary
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Fin du pipeline.
[Pipeline] echo
Analyse SAST terminée avec succès !
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Capture 19 : Console du build #11 (déclenché automatiquement) : scan terminé avec succès, 51 findings, statut SUCCESS.

5.5 Comparatif Webhook GitHub vs SCM Polling

Le déclenchement automatique d'un build peut reposer sur deux approches distinctes : le Webhook, où GitHub notifie activement Jenkins à chaque évènement, ou le SCM Polling, où c'est Jenkins qui vérifie périodiquement l'état du dépôt à intervalle régulier pour détecter une éventuelle modification. Le tableau suivant synthétise les différences principales entre ces deux approches.

Critère	Webhook GitHub	SCM Polling
Temps de réaction	Instantané (en temps réel)	Différé (selon l'intervalle configuré)
Charge réseau / serveur	Très faible (déclenchement sur évènement)	Plus élevée (requêtes répétées)
Prérequis réseau	Jenkins doit être accessible depuis Internet	Fonctionne en réseau local fermé (localhost)
Usage recommandé	Environnement de production / CI-CD réels	Labs d'entraînement locaux / Réseaux privés

Dans le cadre de ce projet, c'est l'approche Webhook qui a été mise en œuvre et testée avec succès, en s'appuyant sur ngrok pour contourner la contrainte d'accessibilité réseau habituellement associée à cette méthode dans un environnement de laboratoire local. Le SCM Polling reste néanmoins une alternative pertinente lorsque Jenkins ne peut pas être exposé publiquement, au prix d'une réactivité moindre et d'une charge réseau plus importante liée aux vérifications périodiques.

6. Étape 4 : Notification par e-mail

Notification automatique de fin de build

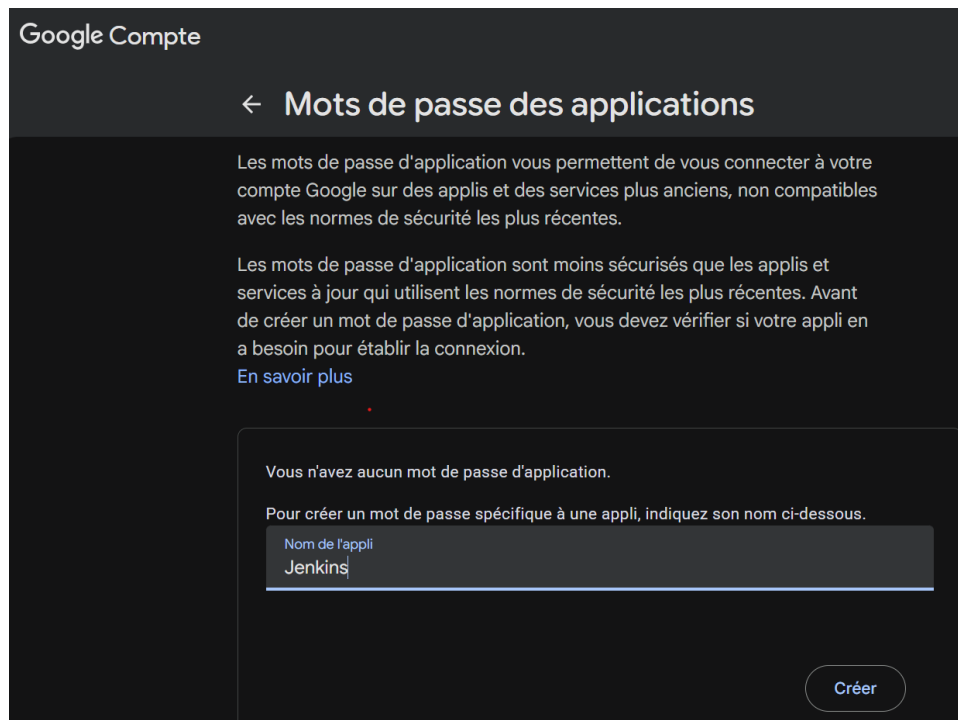
Outils : Jenkins (Email Extension / Mailer), Gmail SMTP

Objectif : Informer automatiquement, par e-mail, de l'issue de chaque build (succès ou échec).

6.1 Contexte

Une fois le build déclenché automatiquement à chaque modification du dépôt, la dernière étape consiste à en être informé sans avoir à consulter manuellement l'interface Jenkins. La solution retenue consiste à configurer Jenkins pour qu'il envoie un e-mail à l'issue de chaque build, via un compte Gmail utilisé comme serveur SMTP sortant.

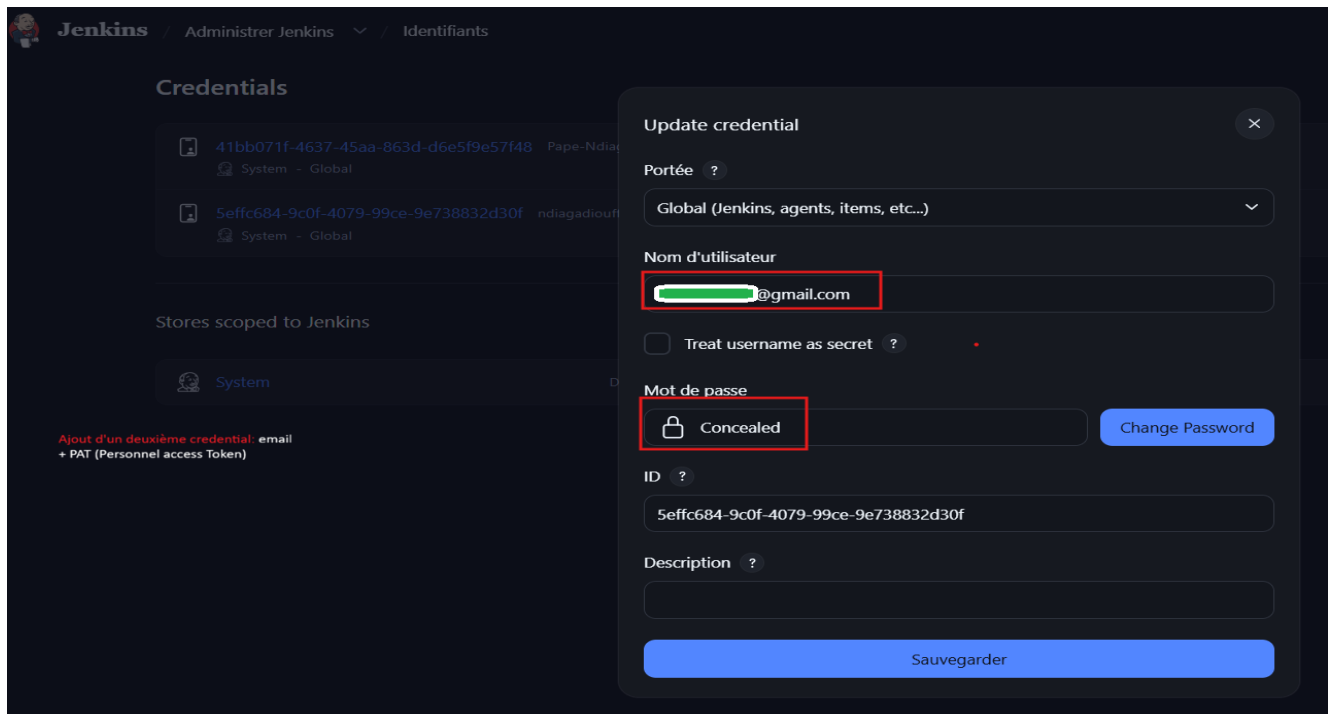
Gmail n'autorisant plus, pour des raisons de sécurité, la connexion directe avec le mot de passe habituel du compte pour ce type d'usage, **un mot de passe d'application** dédié doit être généré depuis les paramètres de sécurité du compte Google, spécifiquement pour Jenkins.



Capture 20 : Création d'un mot de passe d'application Google nommé « Jenkins », destiné à l'authentification SMTP depuis Jenkins.

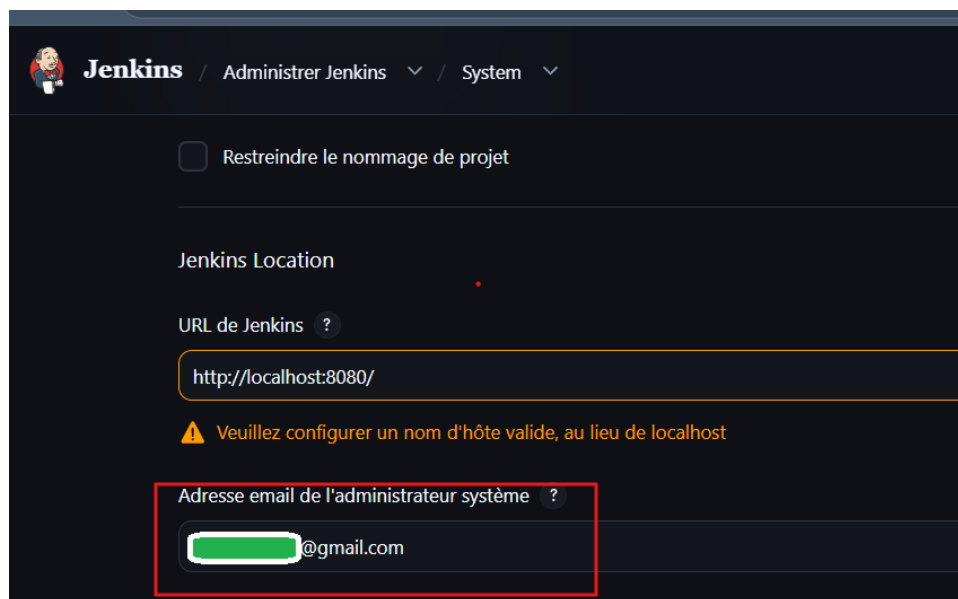
6.2 Configuration des identifiants et de l'hôte Jenkins

Un second identifiant a été ajouté dans le gestionnaire de Credentials de Jenkins, dédié cette fois à l'envoi d'e-mails : il associe l'adresse **Gmail** utilisée comme expéditeur au **mot de passe d'application** généré précédemment.



Capture 21 : Ajout d'un second identifiant Jenkins (adresse e-mail Gmail + mot de passe d'application), distinct de l'identifiant GitHub utilisé pour le SCM.

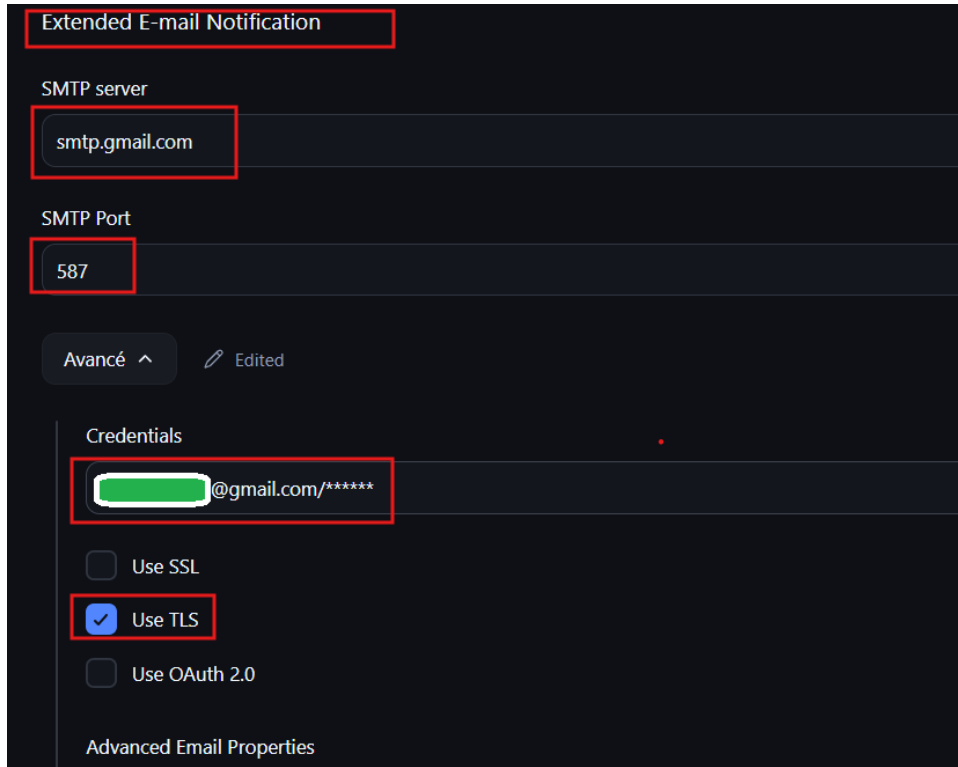
L'adresse e-mail de l'administrateur système a ensuite été renseignée dans la configuration générale de Jenkins (Jenkins Location), afin qu'elle serve d'expéditeur par défaut pour les notifications.



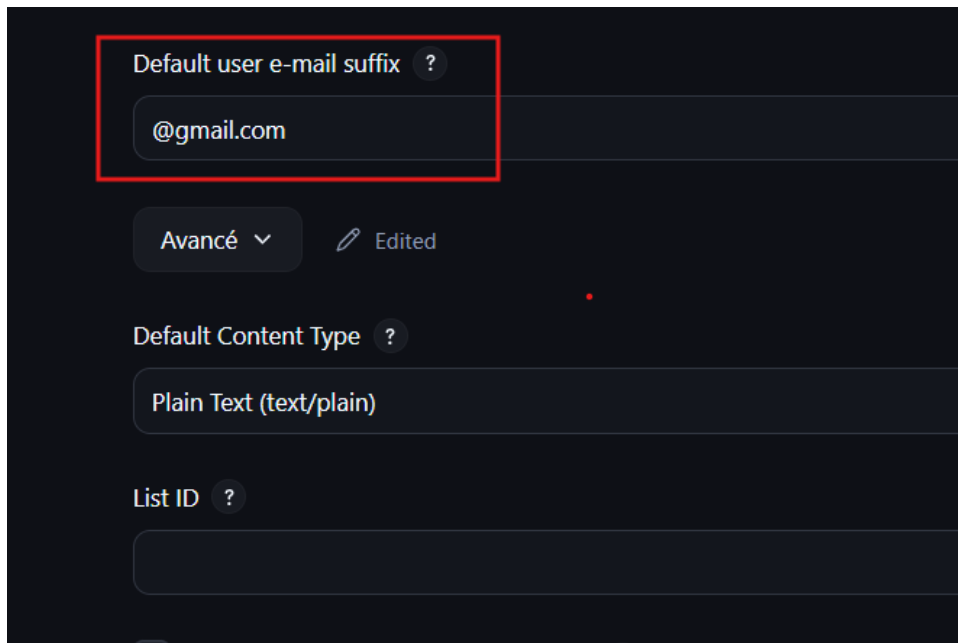
Capture 22 : Configuration de la « Jenkins Location » : URL de l'instance et adresse e-mail de l'administrateur système.

6.3 Configuration du serveur SMTP

Jenkins propose deux mécanismes de notification par e-mail : le module historique « E-mail Notification » et le plugin « Extended E-mail Notification », plus complet, qui a été utilisé ici pour permettre l'envoi du rapport de build en pièce jointe. Les deux ont été configurés en parallèle pour pointer vers le serveur SMTP de Gmail, en HTTPS/TLS sur le port 587, en utilisant l'identifiant créé à l'étape précédente.



Capture 23 : Configuration du plugin Extended E-mail Notification : serveur SMTP smtp.gmail.com, port 587, identifiants associés, option Use TLS activée.



Capture 24 : Paramètres complémentaires : suffixe d'e-mail par défaut (@gmail.com) et type de contenu par défaut (texte brut).

Le mode debug a été activé temporairement afin de faciliter le diagnostic en cas d'échec d'envoi, avant d'être désactivé une fois la configuration validée. Le module historique « E-mail Notification » a été configuré de façon similaire, avec le même serveur SMTP et le même suffixe d'adresse par défaut.

Enable Throttling ?

☒ Enable Debug Mode ?

☐ Require Administrator for Template Testing ?

☐ Enable watching for jobs ?

☐ Allow sending to unregistered users ?

Default Triggers ▾

Content Token Reference ?

Notification par email

Serveur SMTP

smtp.gmail.com

Suffixe par défaut des emails des utilisateurs ?

@gmail.com

Avancé ▴ Edited

Capture 25 : Activation du mode debug du plugin, puis configuration du module historique « Notification par email » (serveur SMTP, suffixe des adresses utilisateurs).

L'authentification SMTP a été activée avec les identifiants Gmail, le chiffrement TLS activé, et le port 587 confirmé pour ce second module.

Suffixe par défaut des emails des utilisateurs ?

@gmail.com

Avancé ▾ Edited

☒ Use SMTP Authentication ?

Nom d'utilisateur

@gmail.com

Mot de passe

Concealed

☐ Utiliser SSL ?

☒ Use TLS

Port SMTP ?

587

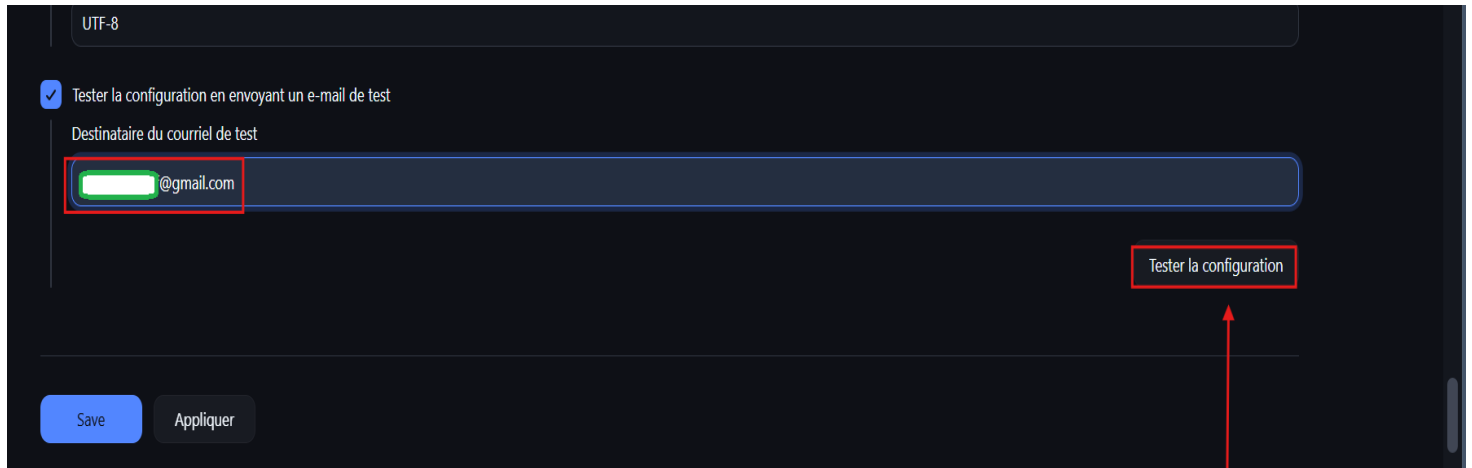
Adresse de réponse

Save Appliquer

Capture 26 : Authentification SMTP activée pour le module historique : nom d'utilisateur, mot de passe (masqué), TLS activé, port 587.

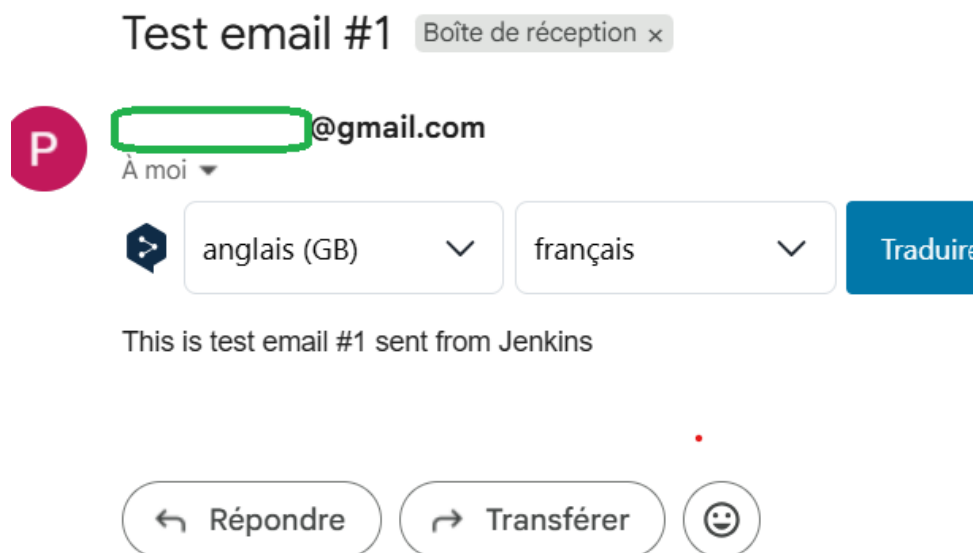
6.4 Test de la configuration

Avant d'intégrer l'envoi d'e-mail dans le pipeline lui-même, la configuration a été validée indépendamment à l'aide de la fonction de test intégrée à Jenkins, qui permet d'envoyer un e-mail de test vers une adresse donnée sans avoir à déclencher un build complet.



Capture 27 : Envoi d'un e-mail de test depuis l'interface de configuration Jenkins, à destination de l'adresse Gmail de l'administrateur.

L'e-mail de test a été reçu avec succès dans la boîte de réception, confirmant que la chaîne d'authentification SMTP (Gmail + mot de passe d'application) fonctionne correctement.



Capture 28 : Réception effective de l'e-mail de test (« Test email #1 sent from Jenkins ») dans la boîte de réception Gmail.

6.5 Intégration dans le pipeline et réception du rapport de build

Une étape supplémentaire (emailxtext) a été ajoutée à la fin du Jenkinsfile, exécutée après l'étape d'analyse Semgrep, afin d'envoyer automatiquement un e-mail récapitulatif à la fin de chaque build, avec le journal complet du build (build.log) joint en pièce jointe. Un build réel du job scan-juice-chop a ensuite été déclenché pour valider l'intégration de bout en bout.

```

pipeline {
    agent any

    environment {
        DEST_EMAIL = 'adresse@gmail.com'
    }

    stages {
        stage('1. Verification Code Source') {
            steps {
                echo "Code source vérifié dans le workspace."
                sh 'ls -la juice-shop'
            }
        }

        stage('2. SAST Scan - Semgrep') {
            steps {
                echo "Lancement du scan de sécurité SAST sur Juice Shop..."
                sh '''
                    docker run --rm \
                    -v devops-infra-jenkins_jenkins-data:/var/jenkins_home \
                    -w /var/jenkins_home/workspace/${JOB_NAME}/juice-shop \
                    semgrep/semgrep semgrep scan --config auto .
                '''
            }
        }
    }

    post {
        always {
            echo "Envoi du rapport par email..."
            emailx (
                to: "${DEST_EMAIL}",
                subject: "Rapport de Scan SAST Jenkins - Job: ${JOB_NAME}
#${BUILD_NUMBER} [${currentBuild.currentResult}]",
                body: ""
            )


Bonjour,



Le build #${BUILD_NUMBER} de la tâche '${JOB_NAME}' est terminé.



- Statut du build : ${currentBuild.currentResult}
- URL du build : ${BUILD_URL}



Vous trouverez ci-joint la sortie console complète contenant les résultats du scan Semgrep.



Cordialement,



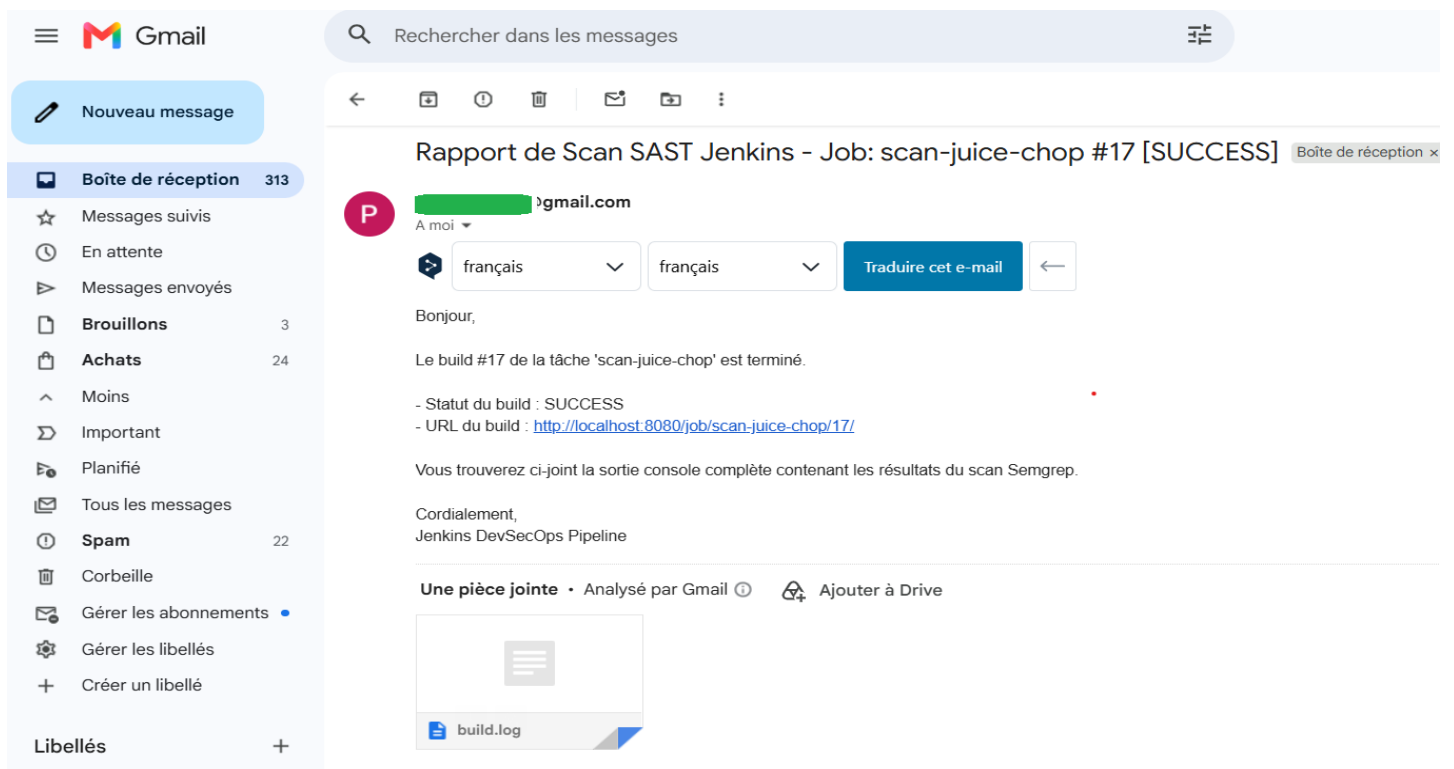
Jenkins DevSecOps Pipeline


```

```

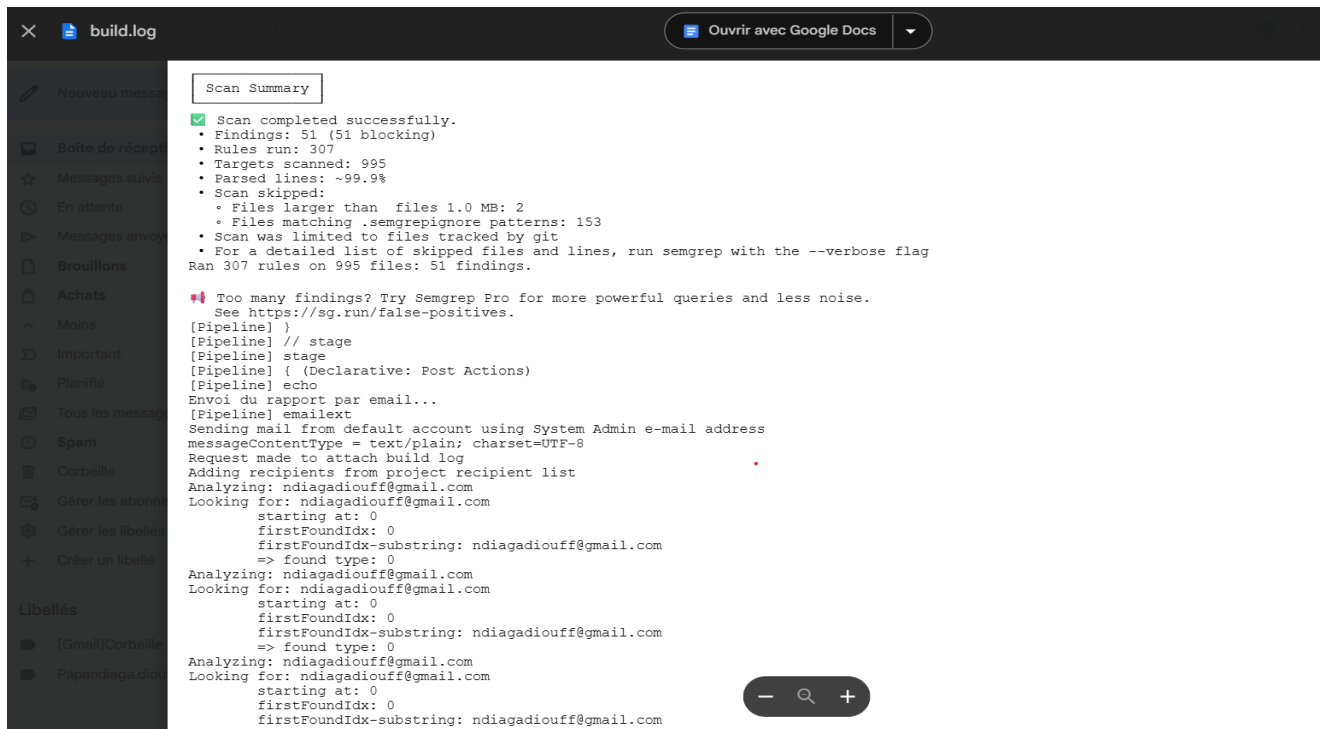
    },
    attachLog: true
  }
}
}

```



Capture 29 : E-mail de notification automatique reçu à l'issue du build #17 : statut SUCCESS, lien direct vers le build, journal complet joint en pièce jointe (build.log).

Le fichier joint contient l'intégralité du journal du pipeline, y compris le résumé du scan Semgrep (51 findings) et la trace de l'étape d'envoi d'e-mail elle-même (résolution du ou des destinataires, confirmation de l'envoi), ce qui permet de conserver une preuve complète et exploitable de chaque exécution directement par e-mail, sans avoir à se connecter à l'interface Jenkins.



Capture 30 : Contenu du fichier build.log joint à l'e-mail : résumé du scan Semgrep et trace de l'étape emailx confirmant l'envoi du rapport.

6.6 Résultat

La chaîne de notification est pleinement fonctionnelle : chaque build, qu'il soit déclenché manuellement ou automatiquement par le Webhook GitHub, se conclut désormais par l'envoi automatique d'un e-mail récapitulatif contenant le statut du build, un lien direct vers celui-ci, et le journal complet en pièce jointe.

Correctif / bonne pratique recommandée : l'usage d'un mot de passe d'application dédié (plutôt que le mot de passe principal du compte Gmail) limite l'impact en cas de compromission des identifiants stockés dans Jenkins, ce mot de passe pouvant être révoqué indépendamment du compte. Il est également recommandé, en environnement de production, de restreindre la liste des destinataires et de chiffrer le canal SMTP (TLS, déjà en place ici) pour éviter toute interception des journaux de build, qui peuvent contenir des informations sensibles sur le code ou l'infrastructure.

7. Conclusion

Les quatre étapes prévues pour la mise en place de cette chaîne d'intégration continue ont été intégralement réalisées et validées :

- Étape 1 : Build Jenkins avec script statique : résolu, scan Semgrep fonctionnel sur du code local, validant le bon fonctionnement de Jenkins et de l'outil d'analyse.
- Étape 2 : Intégration GitHub (Pipeline from SCM) : résolu, Jenkins clone et analyse automatiquement le code source hébergé sur un dépôt GitHub privé, via un jeton d'accès personnel.
- Étape 3 : Déclenchement automatique par Webhook (via ngrok) : résolu, chaque push sur le dépôt GitHub déclenche un build Jenkins en temps réel, sans intervention manuelle.
- Étape 4 : Notification par e-mail : résolu, chaque build se conclut par l'envoi automatique d'un rapport par e-mail (statut, lien, journal complet en pièce jointe), via un serveur SMTP Gmail authentifié par mot de passe d'application.

La combinaison de ces quatre étapes constitue une chaîne DevSecOps complète et autonome : un développeur poussant du code sur le dépôt GitHub déclenche, sans aucune action manuelle, une analyse de sécurité statique du code (SAST) via Semgrep, dont le résultat lui est directement communiqué par e-mail, journal complet à l'appui. Cette automatisation réduit le délai entre l'introduction d'une vulnérabilité dans le code et sa détection, tout en supprimant la charge de supervision manuelle du pipeline.

Ce travail a permis de mettre en pratique, de bout en bout, la construction d'un pipeline d'intégration continue orienté sécurité, depuis l'exécution manuelle d'un script jusqu'à une chaîne entièrement automatisée et notifiée, dans le cadre du module Sécurité des données.